

INSPECTION-RECONCILE

User Guide

Inspection documentation readiness, with a read-only Quickbase adapter

Version	0.1.0
Date	9 October 2026
Audience	Documentation coordinators, QA reviewers and project managers who run assessments and act on the findings
Repository	github.com/bochen2029-pixel/inspection-reconcile

Synthetic demonstration data. Every example in this guide uses the fictional North Creek project (NC-001). No real organization's data appears anywhere in this guide or in the repository. Quickbase is a trademark of Quickbase, Inc.; this project is not affiliated with or endorsed by Quickbase.

Contents

1	About this guide	4
1.1	Who it is for	4
1.2	What you need	4
1.3	Conventions	4
2	What the tool does	5
2.1	The question it answers	5
2.2	Three facts behind the design	5
2.3	What READY_FOR_REVIEW means, and what it never means	5
2.4	What goes in, what comes out	6
3	Quick start	7
3.1	Install	7
3.2	Run the demonstration	7
3.3	Open the results	7
4	Key ideas	9
4.1	Obligations, records and evidence	9
4.2	Outcomes: PASS, FAIL, UNKNOWN, NOT_EVALUATED	9
4.3	Why UNKNOWN is not PASS	9
4.4	Project status	9
4.5	Roots, dependencies and blocked_by	10
4.6	Coverage: how complete is the capture?	10
4.7	Requirement packs	10
5	Running an assessment	11
5.1	From a canonical snapshot	11
5.2	From a Quickbase-shaped export	11
5.3	The summary line and the exit code	11
5.4	What gets written	12
5.5	Output folders and --force	12
5.6	Checking inputs before a run: validate	12
6	Reading the report	14
6.1	The header	14
6.2	What this result means	14
6.3	Coverage	15
6.4	Summary counts	16
6.5	Root findings	16
6.6	The obligation grid	17
6.7	All findings	18
6.8	Provenance	19
6.9	The assessment file	20
7	The eight checks	21
7.1	R0 · Input integrity	21

7.2	R1 · Scope and coverage	21
7.3	R2 · Inspection cardinality	22
7.4	R3 · Identity consistency	23
7.5	R4 · Required artifacts	23
7.6	R5 · Artifact availability	24
7.7	R6 · Approval binding	24
7.8	R7 · Unmatched records (advisory)	26
8	Comparing runs	27
8.1	Running compare	27
8.2	Reading comparison.json	27
9	A guided tour	28
9.1	S02 → S06: a missing inspection, found and fixed	28
9.2	S05: an incomplete capture	29
9.3	S08: an approval that no longer covers the evidence	29
9.4	S12: attachments that were not captured	30
9.5	S17: a Quickbase value the mapping does not know	31
10	Troubleshooting	33
10.1	The command stopped with exit code 2	33
10.2	Common questions	33
11	Glossary	35

1 About this guide

1.1 Who it is for

This guide is for the person who runs an assessment and acts on what it finds: a documentation coordinator preparing a package for review, a QA reviewer checking it, or a project manager who needs to know what still blocks it. It assumes no programming knowledge beyond typing a command in a terminal.

The [Administrator Guide](#) covers installation on a server, the input formats, requirement packs, the Quickbase connection, security and operations.

1.2 What you need

- A computer running Windows, macOS or Linux, with **Python 3.12 or later** and **uv**, a fast Python package manager.
- A copy of the repository.
- For your own projects: a captured *snapshot* or a Quickbase-shaped *export*, and the requirement pack that applies (your administrator provides both). For learning, the repository ships 24 ready-made scenarios.

1.3 Conventions

Commands are shown as you type them from the repository's top folder. A line ending in `\` continues on the next line; in PowerShell, either type the command on one line or end the line with a backtick instead.

```
uv run inspection-reconcile --version          # prints: inspection-reconcile 0.1.0
```

`uv run` runs the tool inside the project's own environment, so nothing is installed system-wide. Names in `CAPITALS_WITH_UNDERSCORES`, such as `NO_CURRENT_INSPECTION`, are reason codes: each one has a fixed meaning, explained in [The eight checks](#).

Note

Every example uses the fictional North Creek project (NC-001), with 40 obligations. The data is synthetic; no real organization's data appears in this guide or in the repository.

2 What the tool does

2.1 The question it answers

`inspection-reconcile` answers one question about a project's inspection documentation: **is this package ready to be reviewed?** It compares the work a project *requires* with the inspection records, documents and approvals that were actually *captured*, and it explains every gap with the source rows that show it.

It is deterministic: the same inputs always give the same verdict, byte for byte, on every operating system. It is also read-only: it never changes a source system.

2.2 Three facts behind the design

1. **Valid rows are not complete work.** Thirty-nine valid inspection records cannot satisfy forty obligations. The expected work therefore comes from an independently accepted **scope**, never from the records being checked.
2. **Absence from a partial view is not absence.** A missing record proves nothing unless the capture is known to be complete. The tool keeps an *established failure* separate from *could not determine*.
3. **Approvals go stale.** An approval covers a specific inspection revision and specific evidence bytes. A later change to either is detected.

2.3 What READY_FOR_REVIEW means, and what it never means

`READY_FOR_REVIEW` means that every required check passed, for the accepted scope and the captured snapshot. The documentation package is complete, consistent and approved as captured, so a reviewer can start.

Important

READY_FOR_REVIEW is not approval. It is not a judgment of physical inspection quality, of regulatory compliance, or of whether the work in the field was done well. A hash proves which bytes were reviewed; it does not prove authorship or truthfulness.

2.4 What goes in, what comes out

goes in	comes out
the accepted scope : the obligations, with a revision and an acceptance record	<code>assessment.json</code> : the verdict and every finding, machine-readable
the inspection records , the artifact inventory (documents, photos) and the evidence files	<code>report.html</code> : a self-contained report for people
the approval decisions , each bound to an inspection revision and to evidence	<code>run-manifest.json</code> : which input files were read, and their digests
a requirement pack : which documents each kind of inspection needs	<code>comparison.json</code> (from <code>compare</code>): what changed between two runs
a snapshot manifest : what was captured, and how complete each part is	

3 Quick start

3.1 Install

```
git clone https://github.com/bochen2029-pixel/inspection-reconcile.git
cd inspection-reconcile
uv sync
```

`uv sync` creates the project environment and installs the two runtime dependencies. It takes a few seconds.

3.2 Run the demonstration

```
uv run inspection-reconcile demo --all --out out/demo
```

The demonstration assesses all 24 fixture scenarios, checks each result against a hand-written expected answer (the *oracle*), and prints one line per scenario:

S01-clean	READY_FOR_REVIEW	matches oracle
S02-missing-inspection	BLOCKED	matches oracle
S03-wrong-project-evidence	BLOCKED	matches oracle
...		
S24-scope-not-accepted	UNKNOWN	matches oracle

It exits with code 0 when every scenario matches its oracle.

3.3 Open the results

Open `out/demo/index.html` in a browser. Each scenario links to its own report.



Figure 1. The demonstration index: every scenario's expected and actual status, with a link to its report

Start with S02, the classic case: one inspection was never recorded.

Tip

The same reports are published online at bochen2029-pixel.github.io/inspection-reconcile, rebuilt from the main branch after every successful test run, so you can browse them before installing anything.

4 Key ideas

4.1 Obligations, records and evidence

- An **obligation** is one piece of required work: an activity (for example `visual_inspection`) on an asset, in a project. The accepted scope lists them; North Creek has 40, `0-001` to `0-040`.
- An **inspection** record says the work was done. It links to its obligation, has a revision, and is either the *current* revision or a superseded one.
- An **artifact** is a document or photo attached to an inspection. Its file lives in the snapshot's evidence folder and is identified by its content digest (SHA-256).
- An **approval** is a reviewer's decision (`approved` , `rejected` or `revoked`) on an inspection revision. It records which evidence it covers, either as one *evidence digest* or as the list of artifact revisions it saw.

4.2 Outcomes: PASS, FAIL, UNKNOWN, NOT_EVALUATED

Every check produces a **finding** with one of four outcomes:

outcome	meaning
PASS	the requirement holds, whatever records might still be missing from the capture
FAIL	the requirement fails, whatever records might still be missing: an established defect
UNKNOWN	the captured data cannot decide; the finding says what would resolve it
NOT_EVALUATED	not run, because a prerequisite did not pass; <code>blocked_by</code> names the prerequisite

4.3 Why UNKNOWN is not PASS

Suppose a capture stopped halfway, and obligation O-037 has no inspection in it. The inspection may simply be among the records that were not captured. Calling that FAIL would be an accusation the data cannot support; calling it PASS would hide a real gap. The honest answer is UNKNOWN, with the reason `ABSENCE_UNCONFIRMED` and the coverage problem that caused it.

The same logic works the other way. One captured, complete inspection proves the work was recorded, but under a partial capture a second, conflicting current inspection could still exist. So "exactly one" stays UNKNOWN (`CARDINALITY_UNCONFIRMED`) until the capture is known to be complete.

4.4 Project status

The project status is computed from the **required** findings only:

status	when
BLOCKED	any required finding is FAIL
UNKNOWN	no FAIL, but some required finding is UNKNOWN or NOT_EVALUATED
READY_FOR_REVIEW	every required finding is PASS

One check, R7 (records outside the scope), is **advisory**: it is reported but never changes the status.

4.5 Roots, dependencies and blocked_by

The checks build on each other. If an obligation has no inspection (R2 fails), there is nothing to check the identity, the documents or the approval of, so R3 to R6 are NOT_EVALUATED, and each names its prerequisite in `blocked_by`.

A **root** is a required FAIL or UNKNOWN that no other finding caused. Roots are what you fix; everything else follows. The report lists roots first, and shows for each one how many findings it blocks.

Tip

Work through the roots, re-capture, and re-run. A dependent finding usually resolves by itself once its root is fixed. The tool shows one problem at a time per obligation: a second, independent problem appears only after the first is fixed.

4.6 Coverage: how complete is the capture?

Every captured dataset (inspections, artifacts, approvals, evidence files) carries a **coverage claim**:

`complete_for_declared_scope`, `partial` or `unverified`, together with the **basis** for the claim, for example `query_total_matched`, `two_pass_stable` and `operator_attestation` for a verified Quickbase read. The requirement pack lists which bases it accepts. A dataset counts as complete only when its claim is complete *and* its basis is accepted *and* nothing in the data contradicts it.

Incomplete coverage turns absences into UNKNOWN rather than FAIL. That is exactly fact 2 above.

4.7 Requirement packs

A requirement pack is a small, versioned YAML file. It says which document kinds each activity needs (North Creek's visual inspections need an `inspection_report` and a `photo`), which coverage bases are acceptable, and how approvals must bind to evidence: `digest` (the content of the files) or `revisions` (their revision labels). The demonstration ships two packs, `policies/north-creek-demo.yml` (digest binding) and `policies/north-creek-revisions.yml`.

5 Running an assessment

5.1 From a canonical snapshot

```
uv run inspection-reconcile assess \  
  --snapshot fixtures/scenarios/S02-missing-inspection/snapshot \  
  --policy policies/north-creek-demo.yml \  
  --out out/s02
```

The command prints one summary line and exits with a code that states the result:

```
BLOCKED roots: FAIL=1 UNKNOWN=0 not_evaluated=4 evaluation_id=sha256:bf6a33fe8f6f...
```

part	meaning
BLOCKED	the project status
roots: FAIL=1 UNKNOWN=0	one root failure, no root unknowns
not_evaluated=4	four required findings were not evaluated, because a prerequisite did not pass
evaluation_id=sha256:bf6a...	the identity of this evaluation: same inputs, same pack, same time, same id

5.2 From a Quickbase-shaped export

An export captured from Quickbase is read through a **field mapping** that says which Quickbase field holds which value. The tool normalizes it into a temporary snapshot, assesses it, and removes the temporary folder:

```
uv run inspection-reconcile assess \  
  --export fixtures/scenarios/S16-quickbase-clean/export \  
  --mapping mappings/quickbase-demo.yml \  
  --policy policies/north-creek-demo.yml \  
  --out out/s16
```

```
READY_FOR_REVIEW roots: FAIL=0 UNKNOWN=0 not_evaluated=0 evaluation_id=sha256:4b0013671cdc...
```

S16 holds the same data as the canonical scenario S01, so it receives the same `evaluation_id`.

5.3 The summary line and the exit code

Scripts and schedulers can act on the exit code alone:

exit code	meaning
0	<code>READY_FOR_REVIEW</code> (<code>assess</code>), no difference (<code>compare</code>), every scenario matched (<code>demo</code>), or success
10	<code>BLOCKED</code>
11	<code>UNKNOWN</code>
2	run error: a usage, configuration or input problem; nothing is written
20	<code>compare</code> found semantic differences
30	<code>demo</code> : at least one scenario did not match its oracle

`assess` always writes its outputs before it exits with 10 or 11. Exit code 1 is never used on purpose.

5.4 What gets written

`--out` names a folder. After a successful run it contains:

file	for	contents
<code>report.html</code>	people	the report described in the next chapter; one self-contained file that opens offline
<code>assessment.json</code>	systems	the status, the coverage table, the counts and every finding, in a fixed order
<code>run-manifest.json</code>	audit	the input files that were read, with their sizes and SHA-256 digests, the tool version and the time of the run

`assessment.json` and `report.html` are byte-identical for the same inputs, on every operating system. Only `run-manifest.json` records when and where the run happened.

5.5 Output folders and `--force`

The `--out` folder must be absent or empty. To re-run into the same folder, add `--force`: the tool then replaces its own output files and leaves anything else there untouched.

```
uv run inspection-reconcile assess --snapshot fixtures/scenarios/S02-missing-inspection/snapshot \
  --policy policies/north-creek-demo.yml --out out/s02 --force
```

Note

`--force` never deletes anything outside `--out`, and never deletes a file it did not write itself.

5.6 Checking inputs before a run: `validate`

`validate` loads every file, checks the configuration, and lists the record-level defects that `assess` would report under R0, without writing anything:

```
uv run inspection-reconcile validate \  
  --snapshot fixtures/scenarios/S23-duplicate-key/snapshot \  
  --policy policies/north-creek-demo.yml
```

```
R0:inspection:INS-004@1#DUPLICATE_KEY FAIL DUPLICATE_KEY required  
assessable: 1 record-level defect(s) would be reported under R0
```

It exits 0 when the inputs can be assessed and 2 when a configuration problem must be fixed first.

6 Reading the report

`report.html` is a single file with no scripts and no external resources: it opens offline, prints cleanly, and can be attached to an email or a ticket as it is. Status colors always come with text labels. The sections appear in the order below.

6.1 The header

SYNTHETIC DATA: fictional project

North Creek (synthetic) (NC-001)

BLOCKED

as of 2026-10-01T18:00:00.000000Z · snapshot NC-001-S02-missing-inspection · scope revision S1 · pack north-creek-demo 3.0.0 · evaluation sha256:bf6a33fe8f6f...

What this result means

- **READY_FOR_REVIEW**: every required check passed for the accepted scope and the captured snapshot.
- **BLOCKED**: at least one established defect (FAIL).
- **UNKNOWN**: no established defect, but the captured data cannot decide (UNKNOWN or NOT_EVALUATED). Absence from a partial view is not absence.

READY_FOR_REVIEW is not approval, and it is not a judgment of physical inspection quality or compliance.

Figure 2. The header of the S02 report, with the meaning of each status

A yellow banner marks synthetic data. Below the project name come the **status**, then one line of context:

- **as of**: the evaluation time. By default it is the end of the capture; `--as-of` sets another.
- **snapshot**: the identifier of the captured snapshot.
- **scope revision**: the accepted scope that defines the expected work.
- **pack**: the requirement pack and its version.
- **evaluation**: the shortened `evaluation_id`. Hover over it for the full value.

6.2 What this result means

This section restates the three statuses in plain language, and that **READY_FOR_REVIEW** is not approval. It is the same on every report, so a reader who receives only the report has the definitions at hand.

6.3 Coverage

SYNTHETIC DATA: fictional project

North Creek (synthetic) (NC-001)

UNKNOWN

as of 2026-10-01T18:00:00.000000Z · snapshot NC-001-S05-incomplete-capture · scope revision S1 · pack north-creek-demo 3.0.0 · evaluation sha256:9eae7769cbe1...

Coverage

dataset	declared	effective	basis	reasons
inspections	partial	partial	extraction_interrupted	COVERAGE_PARTIAL
artifacts	partial	partial	extraction_interrupted	COVERAGE_PARTIAL
approvals	partial	partial	extraction_interrupted	COVERAGE_PARTIAL
evidence_files	partial	partial	extraction_interrupted	COVERAGE_PARTIAL

Summary

root failures: 0 root unknowns: 4 dependent unknowns: 112 not evaluated: 52 advisory (non-pass): 0
findings: 207 (39 pass)

Figure 3. The coverage table and summary of S05, a capture interrupted halfway

One row per dataset:

column	meaning
declared	what the snapshot manifest claims: <code>complete_for_declared_scope</code> , <code>partial</code> or <code>unverified</code>
effective	what the tool accepts after checking the basis and the data; this is the value the checks use
basis	the evidence behind the claim, for example <code>synthetic_universe</code> or <code>query_total_matched</code>
reasons	why the effective value is weaker than claimed, if it is

When `effective` is not `complete_for_declared_scope` , absences in that dataset become UNKNOWN instead of FAIL.

6.4 Summary counts

count	meaning
root failures	required FAILs that nothing else caused: established defects to fix
root unknowns	required UNKNOWNs that nothing else caused: usually a coverage gap or an unreadable value
dependent unknowns	UNKNOWNs that follow from a coverage root; they resolve when the root does
not evaluated	required checks skipped because a prerequisite did not pass
advisory (non-pass)	findings that never change the status, such as R7
findings	the total, with the number that passed

6.5 Root findings

SYNTHETIC DATA: fictional project

North Creek (synthetic) (NC-001)

BLOCKED

as of 2026-10-01T18:00:00.000000Z · snapshot NC-001-S02-missing-inspection · scope revision S1 · pack north-creek-demo 3.0.0 · evaluation sha256:bf6a33fe8f6f...

Root findings

▼ **FAIL** · R2:obligation:0-017 · NO_CURRENT_INSPECTION · blocks 4

Obligation O-017 requires a visual_inspection inspection of asset A-017. No current inspection references it in the inspections dataset, which is complete for scope S1.

field	expected	observed
current_completed_inspections	1	-
current_inspections	-	[]
superseded_revisions	-	0

Resolution: Record or link the inspection for O-017, or remove O-017 from the scope through an accepted scope revision.

Evidence: `scope.csv#row=18`

Figure 4. The root finding of S02: obligation O-017 has no current inspection

Each root is a collapsible panel, FAIL first, then UNKNOWN. The first three are open. Each one shows:

- the **explanation**, built from the actual values;
- an **expected versus observed** table, with the canonical values the check compared;
- the **resolution**: what would fix the finding, or what information would resolve it;
- the **evidence**: locators of the source rows, such as `scope.csv#row=18` , or `table=bsyn00002;rid=117` for a Quickbase record;
- **blocks N**: how many findings depend on this one, directly or through others.

6.6 The obligation grid

Obligations

obligation	R2 cardinality	R3 identity	R4 artifacts	R5 files	R6 approval
0-014	PASS	PASS	PASS	PASS	PASS
0-015	PASS	PASS	PASS	PASS	PASS
0-016	PASS	PASS	PASS	PASS	PASS
0-017	FAIL	N/E	N/E	N/E	N/E
0-018	PASS	PASS	PASS	PASS	PASS
0-019	PASS	PASS	PASS	PASS	PASS
0-020	PASS	PASS	PASS	PASS	PASS

Figure 5. Part of the obligation grid of S02: O-017 fails R2, and its R3 to R6 are not evaluated

One row per obligation, one column per check from R2 to R6. Cells read `PASS` , `FAIL` , `UNK` or `N/E` (not evaluated). Hover over a cell to see the reason code. The grid is the quickest way to see whether a problem is isolated or widespread.

6.7 All findings

All findings

key	outcome	reason	required
R2:obligation:0-014	PASS	SINGLE_CURRENT_COMPLETED	yes
R2:obligation:0-015	PASS	SINGLE_CURRENT_COMPLETED	yes
R2:obligation:0-016	PASS	SINGLE_CURRENT_COMPLETED	yes
R2:obligation:0-017	FAIL	NO_CURRENT_INSPECTION	yes
R2:obligation:0-018	PASS	SINGLE_CURRENT_COMPLETED	yes
R2:obligation:0-019	PASS	SINGLE_CURRENT_COMPLETED	yes
R2:obligation:0-020	PASS	SINGLE_CURRENT_COMPLETED	yes

Figure 6. Part of the findings table of S02

Every finding, in a fixed order: by check (R0 to R7), then by subject. Each row shows the finding's **key**, its outcome, its reason and whether it is required. A key such as `R2:obligation:0-017` reads "check R2, for the obligation O-017". It is stable between runs, which is how `compare` matches findings.

6.8 Provenance

Provenance

evaluation_id	sha256:bf6a33fe8f6fd577096c98065f19f0fc96056d51acfa143a5b139f166eea1409
provenance_id	sha256:cc85e648add415a85d03f59ecbc5a6be43f841d69f9d1fa9bdfd11d14c2925dc
snapshot semantic	sha256:b2fcd906bf7635b983648209a9601be9af27f7aa308705d509020c73e8fe39ce
assessment semantic	sha256:a64a886d1bb0088cbb2fb16ee4f89c8dc6b33c25b7bb8a09eb124ac5fcf9a0e9
policy	sha256:dc5db1b945b0a8f1853d7255c286ec8c499eb3f3e11bec4bc44637825280239
input: approvals	cee27abb200fac53f16859c292f65a3625ab7fbcf29fb5d3871812a031782820
input: artifacts	f7650b60a499f9c704d7f1fef9afa9daa38ce0d824fcb6797df1a814d5cc444b
input: inspections	2beb84ccc889a435ab15fe2e8b0fce927cf3dc15b453630a30a582c7489d8fd0
input: manifest	deffdcdb3125c93062e4b7a14bb7b65f0bf20391c0a500d744079801f6fa2902
input: policy	1748a57c75d1c817e017468d111c35ba3f9414fffa5e435b8a34c2d83638c3a1
input: project	42ab4709072b355df9ddb90068bf177b69376582b36baba5961b936f5b3b0402
input: scope	b6afce18578a3b09a5a400aa821baff54b6889e1f40c27ebdcea0a5c86c1f959

Generated by inspection-reconcile 0.1.0. Quickbase is a trademark of Quickbase, Inc.; this project is not affiliated with or endorsed by Quickbase.

Figure 7. The provenance table of S02

The digests that identify exactly what was assessed:

value	identifies
<code>evaluation_id</code>	the meaning of the run: the engine version, the pack, the snapshot's content and the evaluation time. Equal inputs always give an equal id, even if the rows are shuffled or come from an export
<code>provenance_id</code>	the exact input bytes that were read, as listed in <code>run-manifest.json</code>
snapshot semantic	the snapshot's content alone
assessment semantic	the findings alone; two runs with the same value have the same findings
policy	the requirement pack's content
input: ...	each input file's SHA-256 digest

6.9 The assessment file

`assessment.json` holds everything the report shows, for other systems to consume. Each finding has the same fields in the same order: `key`, `check_id`, `check_type`, `required`, `subject`, `outcome`, `reason`, `expected`, `observed`, `explanation`, `resolution`, `blocked_by`, `caused_by` and `evidence`.

7 The eight checks

Each check is described below with the findings it can produce and what to do about each. The resolution text is the same one the report prints.

7.1 R0 · Input integrity

Are the captured records themselves sound? R0 looks for duplicate keys, conflicting current revisions, invalid values, unsafe file paths and references to records that do not exist. A defective row is **quarantined**: it is reported and kept out of the other checks, never guessed at.

code	outcome	meaning	what to do
INTEGRITY_OK	PASS	no record-level defect anywhere	nothing
DUPLICATE_KEY	FAIL	the same key appears more than once	remove or correct the duplicate records in the source
MULTIPLE_CURRENT_REVISIONS	FAIL	two revisions of one inspection or artifact are both marked current	mark exactly one revision current
COMPLETED_WITHOUT_TIMESTAMP	FAIL	a current, completed inspection has no completion time	record the completion time
INVALID_VALUE	UNKNOWN	a value does not match its type, for example an ID with a trailing space	correct the value in the source, or the mapping
UNMAPPED_VALUE	UNKNOWN	a Quickbase choice the field mapping does not know	confirm the value's meaning and extend the mapping's value map
INVALID_PATH	UNKNOWN	a file path that is unsafe or malformed; the file is never opened	correct the inventory path
MALFORMED_ROW	UNKNOWN	a CSV row with the wrong number of fields	repair the row in the source export
TIMESTAMP_AFTER_AS_OF	UNKNOWN	a completion or decision time later than the evaluation time	check the time zone, the source value or the capture time
DANGLING_REFERENCE	UNKNOWN	a record refers to one that is absent from a dataset declared complete	capture the referenced record, or correct the reference

An R0 finding is **required** when it concerns a record of an in-scope obligation, or a record that cannot be attributed to any obligation. A finding about a record outside the scope is advisory.

7.2 R1 · Scope and coverage

Is there an accepted scope, and how complete is each dataset? R1 produces one finding for the project and one per dataset.

code	outcome	meaning	what to do
SCOPE_ESTABLISHED	PASS	an accepted scope lists the obligations	nothing
SCOPE_MISSING	UNKNOWN	no scope was supplied; nothing can be assessed	supply the accepted scope
SCOPE_NOT_ACCEPTED	UNKNOWN	the scope has no acceptance record	record who accepted the scope, and when
SCOPE_INVALID , SCOPE_EMPTY , SCOPE_REVISION_MISMATCH , SCOPE_PROJECT_MISMATCH	UNKNOWN	the scope rows cannot be identified, are empty, or disagree with the pack or the project	correct the scope, or align its revision with the pack
COVERAGE_COMPLETE	PASS	the dataset is complete for the declared scope, on an accepted basis	nothing
DATASET_MISSING	UNKNOWN	the dataset was not supplied	capture it
COVERAGE_PARTIAL , COVERAGE_UNVERIFIED	UNKNOWN	the capture is declared partial or unverified	capture the dataset completely, with a basis the pack accepts
COVERAGE_BASIS_NOT_ACCEPTED	UNKNOWN	declared complete, on a basis the pack does not accept	re-capture with an accepted basis, or amend the pack deliberately
CHANGED_DURING_CAPTURE	UNKNOWN	records changed while the capture ran	re-capture when the source is stable
COVERAGE_CONTRADICTED , UNATTRIBUTABLE_RECORDS	UNKNOWN	the data contradicts the completeness claim	capture the missing records, or correct the rows

When the project finding is not PASS, every per-obligation check is NOT_EVALUATED: without a scope there is no expected work to compare against.

7.3 R2 · Inspection cardinality

Does each obligation have exactly one current, completed inspection?

code	outcome	meaning	what to do
<code>SINGLE_CURRENT_COMPLETED</code>	PASS	exactly one current inspection, and it is completed	nothing
<code>NO_CURRENT_INSPECTION</code>	FAIL	no current inspection references the obligation, and the inspections are complete	record or link the inspection, or remove the obligation through an accepted scope revision
<code>MULTIPLE_CURRENT</code>	FAIL	two or more current inspections reference it	mark superseded inspections not current, or correct their links
<code>NOT_COMPLETED</code>	FAIL	the current inspection is not completed	complete the inspection, or correct its status
<code>ABSENCE_UNCONFIRMED</code>	UNKNOWN	none captured, but the inspections are not complete	capture the complete inspections dataset
<code>CARDINALITY_UNCONFIRMED</code>	UNKNOWN	one captured, but another could exist uncaptured	capture the complete inspections dataset

7.4 R3 · Identity consistency

Do the inspection and its documents name the obligation's project, asset and activity? A report filed against the wrong asset is caught here.

code	outcome	meaning	what to do
<code>IDENTITY_CONSISTENT</code>	PASS	the inspection and its current documents all match	nothing
<code>INSPECTION_PROJECT_MISMATCH</code> , <code>INSPECTION_ASSET_MISMATCH</code> , <code>INSPECTION_ACTIVITY_MISMATCH</code>	FAIL	the inspection names another project, asset or activity	correct the inspection, or link it to the correct obligation
<code>ARTIFACT_PROJECT_MISMATCH</code> , <code>ARTIFACT_ASSET_MISMATCH</code>	FAIL	an attached document names another project or asset	attach the correct document, or correct the artifact
<code>IDENTITY_UNCONFIRMED</code>	UNKNOWN	no conflict found, but the artifacts are not complete	capture the complete artifacts dataset

7.5 R4 · Required artifacts

Is there a current document of every kind the pack requires?

code	outcome	meaning	what to do
<code>REQUIRED_KINDS_PRESENT</code>	PASS	every required kind is present	nothing
<code>KIND_MISSING</code>	FAIL	a required kind is missing, and the artifacts are complete	attach the missing document, or correct its kind or current flag
<code>KIND_ABSENCE_UNCONFIRMED</code>	UNKNOWN	a required kind was not captured, but the artifacts are not complete	capture the complete artifacts dataset
<code>REQUIREMENT_UNDEFINED</code>	UNKNOWN	the pack defines no documents for this activity	add requirements for the activity to the pack

7.6 R5 · Artifact availability

Can every required file be read and hashed? File names are matched exactly, including letter case, on every operating system, so a result never depends on where the tool runs.

code	outcome	meaning	what to do
<code>EVIDENCE_AVAILABLE</code>	PASS	every required file was read and hashed	nothing
<code>FILE_ABSENT</code>	FAIL	a required file is not in the evidence, which is complete	restore the file, or correct the inventory path
<code>NOT_CAPTURED</code>	UNKNOWN	a required file was not captured; the evidence is partial	capture the file, or confirm its absence in the source
<code>FILE_UNREADABLE</code>	UNKNOWN	the file exists but could not be read	fix the file's permissions or format, then re-run
<code>FILE_TOO_LARGE</code>	UNKNOWN	the file exceeds the pack's size limit	raise the limit deliberately, or review the file separately
<code>AVAILABILITY_UNCONFIRMED</code>	UNKNOWN	every captured file was available, but other required artifacts could exist	capture the complete artifacts dataset

When a file is absent only because of letter case (`Photo.png` against `photo.png`), the explanation says so: "A file differing only in letter case exists".

7.7 R6 · Approval binding

Does the latest approval cover the current inspection revision and the current evidence?

code	outcome	meaning	what to do
<code>APPROVAL_COVERS_CURRENT</code>	PASS	the latest decision approves the current revision and evidence	nothing
<code>NO_APPROVAL</code>	FAIL	there is no decision at all, and the approvals are complete	obtain a review decision

code	outcome	meaning	what to do
REVISION_NOT_APPROVED	FAIL	approvals exist, but only for older revisions	obtain approval for the current revision
EVIDENCE_CHANGED_SINCE_APPROVAL	FAIL	the decisions bind different evidence: it changed after approval	re-review the current evidence, or restore the approved version
APPROVAL_REJECTED , APPROVAL_REVOKED	FAIL	the latest decision on the current evidence is a rejection or a revocation	resolve the reviewer's decision
APPROVAL_UNCONFIRMED	UNKNOWN	the approvals are not complete; a later decision could exist	capture the complete approvals dataset
BINDING_UNAVAILABLE	UNKNOWN	the approval does not record which evidence it covers	record the evidence digest, or the approved artifact revisions
BINDING_STRENGTH_INSUFFICIENT	UNKNOWN	the approval binds revisions only, but the pack requires a digest	record the evidence digest, or use a pack that accepts revision binding
CONFLICTING_DECISIONS	UNKNOWN	decisions with the same timestamp disagree	correct the decision records

Digest binding and revision binding. With **digest** binding, the default, an approval stores one *evidence digest*: a SHA-256 over the content of every required file. Change one byte of one photo and the digest no longer matches. With **revision** binding, an approval lists the artifact revisions it saw. That is weaker: a file replaced under an unchanged revision label goes unnoticed. Scenarios S08 and S18 show the difference.

The digest an approval should store. `evidence-digest` prints it, together with the files it covers:

```
uv run inspection-reconcile evidence-digest \
  --snapshot fixtures/scenarios/S01-clean/snapshot \
  --policy policies/north-creek-demo.yml --inspection INS-001
```

```
sha256:4d97b90411ac5ee1c2e0f1564a147d0b3d84c3fe16d4c9bed7ccbb8c983e63d6
[
  {
    "artifact_id": "ART-001-P",
    "document_kind": "photo",
    "revision": "1",
    "sha256": "14e2f9ba2bf3bfecf24618d6be0387d667f7a3ed932ddaceefb0f5a1f6d90c99"
  },
  ...
]
```

A review process that stores this value with its decision gets byte-level protection: any later change to the evidence is detected as `EVIDENCE_CHANGED_SINCE_APPROVAL`.

7.8 R7 · Unmatched records (advisory)

Are there current inspections outside the accepted scope? An inspection that points at no in-scope obligation often means the scope is stale. R7 reports it as `UNMATCHED_INSPECTION`, which is advisory: it never changes the status. `NO_UNMATCHED_RECORDS` means every captured current inspection references an in-scope obligation.

8 Comparing runs

8.1 Running compare

`compare` matches the findings of two assessments by key and reports what changed. Each argument is an `assessment.json` or a folder that contains one:

```
uv run inspection-reconcile compare --before out/s02 --after out/s06
```

```
BLOCKED -> READY_FOR_REVIEW  changed=5 added=0 removed=0 unchanged=202
```

It exits 0 when nothing changed and 20 when something did, so a script can tell a fixed package from an unchanged one. A finding counts as **changed** when its outcome, reason, expected or observed values, `blocked_by`, `caused_by` or requiredness differ. Explanation texts and evidence locators are not compared.

8.2 Reading comparison.json

`--out FILE` writes the comparison as JSON:

```
uv run inspection-reconcile compare --before out/s02 --after out/s06 --out out/compare.json
```

```
{
  "schema": "inspection-reconcile/comparison/v1",
  "before": {"evaluation_id": "sha256:bf6a...", "status": "BLOCKED"},
  "after": {"evaluation_id": "sha256:eda9...", "status": "READY_FOR_REVIEW"},
  "added": [],
  "removed": [],
  "changed": [
    {"key": "R2:obligation:0-017",
      "before": {"outcome": "FAIL", "reason": "NO_CURRENT_INSPECTION"},
      "after": {"outcome": "PASS", "reason": "SINGLE_CURRENT_COMPLETED"}},
    ...
  ],
  "unchanged_count": 202
}
```

- `added` and `removed` list keys present in only one of the two assessments.
- `changed` lists each changed finding's outcome and reason, before and after.
- `unchanged_count` counts the rest.

An existing `--out` file is replaced only with `--force`, and `compare` refuses to write over either of its inputs.

9 A guided tour

Five scenarios show the tool's behavior from start to finish. Each command below was run as printed.

9.1 S02 → S06: a missing inspection, found and fixed

In S02, obligation O-017 (a visual inspection of asset A-017) has no inspection record at all. The inspections dataset is complete, so the absence is an established defect.

```
uv run inspection-reconcile assess --snapshot fixtures/scenarios/S02-missing-inspection/snapshot \
  --policy policies/north-creek-demo.yml --out out/s02 # exit 10: BLOCKED
```

The report has a single root failure, `R2:obligation:O-017 · NO_CURRENT_INSPECTION · blocks 4`. Its R3 to R6 are NOT_EVALUATED, because there is no inspection to check. S06 is the same project after the inspection was recorded:

```
uv run inspection-reconcile assess --snapshot fixtures/scenarios/S06-corrected/snapshot \
  --policy policies/north-creek-demo.yml --out out/s06 # exit 0:
READY_FOR_REVIEW
uv run inspection-reconcile compare --before out/s02 --after out/s06 # exit 20
```

SYNTHETIC DATA: fictional project

North Creek (synthetic) (NC-001)

READY_FOR_REVIEW

as of 2026-10-02T18:00:00.000000Z · snapshot NC-001-S06-corrected · scope revision S1 · pack north-creek-demo 3.0.0 · evaluation sha256:eda920e1325e...

Summary

root failures: 0 root unknowns: 0 dependent unknowns: 0 not evaluated: 0 advisory (non-pass): 0
findings: 207 (207 pass)

Root findings

No root findings.

Figure 8. The S06 report: no root findings, and the project is ready for review

`compare` shows exactly five changed findings, R2 to R6 for O-017, and 202 unchanged. Fixing the root resolved everything that depended on it.

9.2 S05: an incomplete capture

S05's capture was interrupted: every dataset is declared `partial` on the basis `extraction_interrupted`.

```
uv run inspection-reconcile assess --snapshot fixtures/scenarios/S05-incomplete-capture/snapshot \
  --policy policies/north-creek-demo.yml --out out/s05                                # exit 11: UNKNOWN
```

```
UNKNOWN roots: FAIL=0 UNKNOWN=4 not_evaluated=52 evaluation_id=sha256:9eae7769cbe1...
```

The four roots are the coverage findings, one per dataset. Obligations whose inspections were not captured get `ABSENCE_UNCONFIRMED`, not FAIL: they may well be among the records the capture missed. The other 112 unknowns are dependent; they resolve once a complete capture replaces this one.

9.3 S08: an approval that no longer covers the evidence

In S08, the photo for O-009 was replaced after approval, under the same revision label. The approval's evidence digest no longer matches the current bytes.

```
uv run inspection-reconcile assess --snapshot fixtures/scenarios/S08-silent-byte-change/snapshot \
--policy policies/north-creek-demo.yml --out out/s08 # exit 10: BLOCKED
```



Figure 9. The S08 root finding: the evidence changed after approval

The root is `R6:obligation:0-009 · EVIDENCE_CHANGED_SINCE_APPROVAL`. The expected-versus-observed table shows the digest the approval stored next to the digest of the current files. Under revision binding (scenario S18) the same change cannot be seen, which is why digest binding is the default.

9.4 S12: attachments that were not captured

In S12, the capture skipped one attachment on purpose, and the manifest says so: the evidence files are `partial`, on the basis `attachment_capture_skipped`.

```
uv run inspection-reconcile assess --snapshot fixtures/scenarios/S12-attachment-not-captured/snapshot \
    --policy policies/north-creek-demo.yml --out out/s12 # exit 11: UNKNOWN
```

SYNTHETIC DATA: fictional project

North Creek (synthetic) (NC-001)

UNKNOWN

as of 2026-10-01T18:00:00.000000Z · snapshot NC-001-S12-attachment-not-captured · scope revision S1 · pack north-creek-demo 3.0.0 · evaluation sha256:6377d63c8c25...

Coverage

dataset	declared	effective	basis	reasons
inspections	complete_for_declared_scope	complete_for_declared_scope	synthetic_universe	-
artifacts	complete_for_declared_scope	complete_for_declared_scope	synthetic_universe	-
approvals	complete_for_declared_scope	complete_for_declared_scope	synthetic_universe	-
evidence_files	partial	partial	attachment_capture_skipped	COVERAGE_PAF

Root findings

▼ UNKNOWN · R1:dataset:evidence_files · COVERAGE_PARTIAL · blocks 0

Dataset evidence_files is declared partial (basis: attachment_capture_skipped).

field	expected	observed
basis	-	["attachment_capture_skipped"]
consistency	-	"not_applicable"
declared	-	"partial"
effective	"complete_for_declared_scope"	"partial"
reasons	-	["COVERAGE_PARTIAL"]

Resolution: Capture evidence_files completely for the declared scope.

Figure 10. S12: the evidence coverage is partial, so the missing file is NOT_CAPTURED rather than FILE_ABSENT

The root is the coverage finding for `evidence_files`. The missing file gives R5 `NOT_CAPTURED` for O-019, an UNKNOWN caused by that root, rather than `FILE_ABSENT`, a FAIL. The tool does not accuse a file it was never shown.

9.5 S17: a Quickbase value the mapping does not know

S17 is a Quickbase-shaped export in which one inspection's status reads `Completed - pending QA`. That choice is not in the mapping's value map. The tool does not guess what it means:

```
uv run inspection-reconcile assess --export fixtures/scenarios/S17-quickbase-unmapped-
value/export \
  --mapping mappings/quickbase-demo.yml --policy policies/north-creek-demo.yml --out out/s17
# exit 11
```

SYNTHETIC DATA: fictional project

North Creek (synthetic) (NC-001)

UNKNOWN

as of 2026-10-01T18:00:00.000000Z · snapshot NC-001-S17-quickbase-unmapped-value · scope revision S1 · pack north-creek-demo 3.0.0 · evaluation sha256:cdc548715985...

Root findings

▼ UNKNOWN · R0:inspection:INS-006@1#UNMAPPED_VALUE · UNMAPPED_VALUE · blocks 5

Inspection INS-006 revision 1: field completion_status value "Completed - pending QA" violates unmapped; the mapping has no entry for these source values.

field	expected	observed
count	-	1
violations	-	[{ "field": "completion_status", "rule": "unmapped", "value": "Completed - pending QA" }]

Resolution: Confirm the values' meaning and extend the mapping's value map.

Evidence: table=bsyn00002;rid=106

Figure 11. S17: an unmapped Quickbase choice is reported, never guessed

The root is `R0:inspection:INS-006@1#UNMAPPED_VALUE`. The fix belongs to whoever owns the mapping: confirm what the new choice means, then extend the value map. Until then, O-006's checks are NOT_EVALUATED.

10 Troubleshooting

10.1 The command stopped with exit code 2

Exit code 2 is a run error: the tool wrote nothing and printed one diagnostic line, `error: CODE: message`. The most common ones:

code	cause	what to do
<code>OUT_NOT_EMPTY</code>	the <code>--out</code> folder already holds files	use a new folder, or add <code>--force</code> to replace the tool's own outputs
<code>OUT_EXISTS</code>	<code>compare --out</code> or <code>export-sqlite --out</code> names an existing file	choose another file name, or add <code>--force</code>
<code>USAGE</code>	a missing or conflicting option, for example <code>--export</code> without <code>--mapping</code>	correct the command; <code>--help</code> lists the options
<code>POLICY_NOT_EFFECTIVE</code>	the evaluation time is before the pack's <code>effective_from</code> date	use the pack that was in force, or a later <code>--as-of</code>
<code>PROJECT_MISMATCH</code>	the snapshot's project is not the pack's project	use the pack for this project
<code>CONFIG_INVALID</code> , <code>CONFIG_UNREADABLE</code>	a pack, mapping or manifest has an unknown member or a wrong value, or cannot be read	read the message: it names the file and the member
<code>CSV_HEADER_INVALID</code> , <code>CSV_UNREADABLE</code>	a CSV file has the wrong columns, or is not UTF-8	fix the export, then capture again
<code>COMPARE_INPUT_INVALID</code>	a <code>compare</code> argument is not an assessment	point it at an <code>assessment.json</code> or at an output folder

The [Administrator Guide](#) lists every code.

10.2 Common questions

Nothing failed, so why is the status UNKNOWN? Some required finding could not be decided, usually because a dataset is not complete. Look at the coverage table and the root unknowns: each one says what information would resolve it.

Why are so many checks NOT_EVALUATED? They depend on a root that did not pass. Fix the root and re-run; most of them resolve by themselves.

The file is there, but the report says FILE_ABSENT. Compare the letter case of the inventory path with the actual file name. The tool matches names exactly on every operating system, and the explanation adds a hint when only the case differs.

The approval exists, but R6 fails. Look at the expected-versus-observed table. Either the approval covers an older revision (`REVISION_NOT_APPROVED`), or the evidence changed after it was approved (`EVIDENCE_CHANGED_SINCE_APPROVAL`).

The report says READY_FOR_REVIEW. Is the project approved? No. The documentation is complete and consistent for the captured snapshot, so the review can start. Approval is a person's decision.

Can I feed the output to another system? Yes: use `assessment.json` and the exit code. Use `--log-json` to get diagnostics on standard error as JSON lines.

11 Glossary

term	meaning
accepted scope	the list of obligations, with a revision and a record of who accepted it, and when
advisory	a finding that is reported but never changes the status (R7, and R0 findings outside the scope)
artifact	a document or photo attached to an inspection, identified by its content digest
basis	the evidence behind a coverage claim, such as <code>query_total_matched</code>
blocked_by	the prerequisite findings that kept a check from running
caused_by	the coverage finding behind an UNKNOWN
coverage	how complete a captured dataset is, as declared and as effectively accepted
evidence digest	a SHA-256 over the content of an inspection's required files; what an approval should store
evaluation_id	the identity of an evaluation: the engine, the pack, the snapshot's content and the evaluation time
finding	the result of one check for one subject: an outcome, a reason, values, an explanation and a resolution
obligation	one piece of required work: an activity on an asset in a project
oracle	the hand-written expected result of each fixture scenario
requirement pack	the versioned YAML file that says what each activity requires
root	a required FAIL or UNKNOWN that no other finding caused
snapshot	a captured set of scope, records and evidence files, described by a manifest